Sentiment Classifier

Structured Notes

RNN Limitations, Vanishing & Exploding Gradients, Memory Compression & Fading, LSTM & GRU Introduction

Contents

1 Recap: RNN Training on Short Sequences **2**

2 RNN and the Long Sequence Problem **2**
2.1 Testing Long-Term Dependency . . . 2
2.2 Experimental Observation . . . 2

3 Why Does RNN Struggle with Long Sequences? **3**
3.1 Hidden State Transformation . . . 3
3.2 Why tanh and Not ReLU? . . . 3

4 Vanishing Gradient Problem **3**
4.1 Intuition . . . 4
4.2 Consequences of Vanishing Gradient . . . 4
4.3 Vanishing Gradient is a Global Problem . . . 4

5 Exploding Gradient Problem **4**
5.1 Intuition . . . 4
5.2 Consequences of Exploding Gradient . . . 5
5.3 Comparison: Vanishing vs Exploding Gradient . . . 5

6 Why Basic RNN Fails: Core Problems **5**
6.1 Problem 1: Memory Compression . . . 5
6.2 Problem 2: Memory Fading . . . 5
6.3 Problem 3: Gradient Problems . . . 6
6.4 Sentiment Analysis Failure Example . . . 6

7 Solution: Gated Memory (LSTM and GRU) **6**
7.1 Motivation: Human Memory Analogy . . . 6
7.2 Gated Memory Concept . . . 6
7.3 LSTM (Long Short-Term Memory) . . . 7
7.4 GRU (Gated Recurrent Unit) . . . 7
7.5 Smart Memory Demonstration . . . 7

8 Summary **8**

1 Recap: RNN Training on Short Sequences

The RNN model was previously implemented from scratch and trained using the `nn.Module` interface in PyTorch. The model successfully learned short sequences — for example, fitting a simple dummy model (analogous to $y = x^2$ in ANN) achieved near 100% accuracy because the sequence was very simple and short.

Key Observation

Basic RNN works well on short sequences. The critical question is: **Can RNN remember long sequences?**

2 RNN and the Long Sequence Problem

2.1 Testing Long-Term Dependency

Consider a sentiment classification task with the sequence: **"good bad bad good"**. The correct output is **positive** because the first and last words are "good." For the model to classify this correctly, it must:

- Remember the first word across the entire sequence
- Carry memory across the long sequence
- Combine distant information to make a final prediction

Long-Term Dependency

A **long-term dependency** is a dependency between distant elements in a sequence. The model needs to connect information that appeared much earlier in the sequence with information that appears later to produce a correct output.

2.2 Experimental Observation

When testing the trained RNN on longer sequences:

- Predictions become **fluctuating** and **unstable**
- The RNN **struggles** with the long sequence
- The model becomes **uncertain** for long sequences

Testing Long Sequence Prediction

```
# Encode the test sequence
sequence = ["good", "bad", "bad", "good"]
encoded = torch.tensor([word_to_index[w] for w in sequence])

# Predict with no gradient (inference mode)
with torch.no_grad():
    output = model(encoded)
    prediction = torch.argmax(output)
```

Listing 1: Predicting on a long sequence with RNN

Why torch.no_grad()?

During testing and prediction, gradients are not required. Gradient tracking is only needed during training for weight updates. Using `torch.no_grad()` disables gradient computation, saving memory and computation.

3 Why Does RNN Struggle with Long Sequences?

The fundamental reason is the **vanishing gradient problem**. As memory gradually fades across time steps, the model loses its ability to retain earlier information.

3.1 Hidden State Transformation

The RNN updates its hidden state at every time step. Over time, information must pass through many transformations:

- **Step 1:** Information enters the hidden state
- **Step 2:** Hidden state transforms
- **Step 3:** Transforms again
- **Step 10:** Eventually, original information becomes very small

As the sequence length increases, the number of time steps increases. Each time step applies a transformation to the hidden state, and the same single RNN architecture is called repeatedly (not different layers).

RNN Hidden State Update:

$$h_t = \tanh(W_{hx} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$$

Where:

- h_t is the hidden state at time step t
- W_{hx} is the input-to-hidden weight matrix
- W_{hh} is the hidden-to-hidden weight matrix
- x_t is the input at time step t
- h_{t-1} is the previous hidden state
- b is the bias term

The tanh function converts everything to the range $[-1, 1]$. Repeatedly applying this transformation causes the information to compress and fade.

3.2 Why tanh and Not ReLU?

The tanh activation function is used in RNN hidden states instead of ReLU because:

- ReLU is **not zero-centered** — it is always increasing for positive inputs
- ReLU can cause **gradient explosion** because it gives unbounded positive values
- The hidden state needs values centered around a point, which tanh provides with its $[-1, 1]$ range
- Leaky ReLU is an update of ReLU that gives very small weights to negative values, but is still not ideal for RNN because it is not centered
- Leaky ReLU is good for CNN but not suitable for RNN hidden state updates

4 Vanishing Gradient Problem

Vanishing Gradient

The **vanishing gradient problem** occurs when gradients become extremely small (close to zero) during backpropagation. This prevents effective weight updates, causing the network to stop learning long-term dependencies.

4.1 Intuition

Imagine multiplying small numbers repeatedly:

$$0.5 \times 0.5 \times 0.5 \times 0.5 \times 0.5 = 0.5^5 = 0.03125$$

Since tanh produces values between -1 and 1, repeated multiplication of these values drives the gradient toward zero very quickly.

Vanishing Gradient Demonstration

```python
import numpy as np

gradient = 0.5
for step in range(1, 11):
    gradient *= 0.5
    print(f"Step {step}: gradient = {gradient:.6f}")

# Step 1:  gradient = 0.250000
# Step 5:  gradient = 0.015625
# Step 10: gradient = 0.000488
# The gradient vanishes very quickly!
```

Listing 2: Gradient shrinking over time steps

4.2 Consequences of Vanishing Gradient

- **Memory fading:** Early information in the sequence is lost
- **Learning difficulty:** Weights stop updating effectively because gradient signals become too weak
- **Long-term dependency failure:** The model cannot connect distant elements in a sequence

Interview Importance

The vanishing gradient problem is a frequently asked question in deep learning interviews. It is one of the major limitations in deep learning that all practitioners should understand thoroughly.

4.3 Vanishing Gradient is a Global Problem

The vanishing gradient problem is not unique to RNN. It occurs in any neural network where backpropagation is performed repeatedly in a very repeated manner. However, it is particularly severe in RNNs because the same architecture is applied at every time step.

5 Exploding Gradient Problem

Exploding Gradient

The **exploding gradient problem** is the opposite of vanishing gradient. During backpropagation, if gradient values are greater than 1, repeated multiplication causes gradients to become very large, leading to unstable training.

5.1 Intuition

Consider multiplying values greater than 1 repeatedly:

$$1.5 \times 1.5 \times 1.5 \times 1.5 \times 1.5 = 1.5^5 = 7.59375$$

This leads to very large gradients, causing training instability.

5.2 Consequences of Exploding Gradient

- **Training instability:** Loss curve shows sudden spikes
- **Model divergence:** Loss becomes very large
- **Weight instability:** Weights become unstable and overflow
- **Missing minima:** Loss overshoots, causing the optimizer to miss the optimal minimum

5.3 Comparison: Vanishing vs Exploding Gradient

Aspect	Vanishing Gradient	Exploding Gradient
Gradient value	Approaches zero	Grows very large
Cause	Multiplication of values < 1	Multiplication of values > 1
Effect on learning	Weights stop updating	Weights become unstable
Effect on loss	Learning stalls	Loss spikes and diverges
Handling difficulty	Harder to handle in RNN	Easier to handle

Handling in RNN vs CNN

Exploding gradient is much easier to handle than vanishing gradient. In CNN, both problems are easier to handle because the activation function can be changed (e.g., using Leaky ReLU). In RNN, these are harder to handle because the activation function (tanh) cannot simply be replaced — it is essential for the recurrent structure.

6 Why Basic RNN Fails: Core Problems

6.1 Problem 1: Memory Compression

Memory Compression

Memory compression occurs because the hidden state has a fixed size, while the sequence length can be arbitrarily long. The model must compress all sequential information into a fixed number of neurons.

Example: If the hidden state size is 8 neurons but the sequence length is 50, the model must compress 50 time steps of information into just 8 numbers. This inevitably causes information loss.

Key points:

- The hidden state size is **fixed** — it does not change dynamically
- The time step update is dynamic (longer sequences have more time steps)
- But the model structure (hidden state size) remains constant
- No model exists that can automatically increase its structure based on sequence length or complexity

6.2 Problem 2: Memory Fading

The hidden state update is a **recursive function**. Information gets transformed repeatedly at each time step. Eventually, early information disappears through this repeated transformation.

Memory Fading Demonstration

```
import torch
import numpy as np

# Initial hidden memory with 4 neurons
memory = np.array([0.8, -0.6, 0.9, -0.7])
```

```
print(f"Initial memory: {memory}")

for step in range(1, 6):
    memory = np.tanh(memory)  # Apply tanh transformation
    print(f"After step {step}: {memory}")

# Memory values decrease toward zero at each step
# This demonstrates exponential decay of information
```

Listing 3: Demonstration of how memory fades with repeated tanh

The memory values decrease at each step, demonstrating **exponential decay** of information. After repeated application of tanh, the values converge toward zero.

6.3 Problem 3: Gradient Problems

During training, gradients flow backward through time. The repeated multiplication in $h_t \rightarrow h_{t-1}$ causes either vanishing gradient or exploding gradient. This is known as **Backpropagation Through Time (BPTT)**.

6.4 Sentiment Analysis Failure Example

Consider the sentence: *"The movie was boring at first but became amazing later."*

- From a human perspective, the sentiment is **positive** (or neutral to positive) because "amazing" in the end modifies the overall impression
- Basic RNN may focus only on "boring" because early memory fades away, or conversely, may focus only on "amazing" (the last word) while losing the early context
- The correct sentiment requires understanding both "boring" and "amazing" and their positions in the sentence

Another example: *"The first half was horrible."* This should be classified as negative. But if "amazing" appears at the end, basic RNN would likely classify it as positive because it struggles with long-term memory.

Key Insight

Basic RNN does not selectively remember — everything is treated equally. There is **no control over memory** and **no selective remembering**. The tanh transformation equation is applied uniformly at every step, giving more weight to the most recent input.

7 Solution: Gated Memory (LSTM and GRU)

7.1 Motivation: Human Memory Analogy

Human memory does not remember everything. Instead, it:

- **Remembers** important things (e.g., lending money to someone, even from a year ago)
- **Forgets** unimportant things (e.g., what clothes were worn a week ago)
- **Updates** memory when needed

The goal is to make neural networks behave similarly — selectively remembering important information and forgetting unimportant information.

7.2 Gated Memory Concept

Gated Memory

Instead of simple memory update (as in basic RNN), **gated memory** uses controlled memory updates. The model **learns** what to remember, what to forget, and what to update. This is the key idea behind LSTM and GRU.

7.3 LSTM (Long Short-Term Memory)

LSTM introduces a **memory cell** plus **three gates**:

1. **Forget Gate:** Decides what information to discard from the cell state
2. **Input Gate:** Decides what new information to store in the cell state
3. **Output Gate:** Decides what information to output from the cell state

These gates control memory flow and help preserve long-term information. The name "Long Short-Term Memory" reflects its purpose: deciding which memory should go into **long-term** storage and which should remain in **short-term** storage.

Example with sentiment analysis: In the sentence "The movie was boring at first but became amazing later," the word "boring" should be stored in long-term memory while "amazing" remains in short-term memory. Based on both, the model can make an informed sentiment classification.

Skip Connection Analogy

The LSTM memory update behaves like a **skip connection**. The memory update formula is:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

This additive structure (similar to skip connections in ResNets) prevents the memory from fading away, as demonstrated by the fact that memory values remain stable across steps.

7.4 GRU (Gated Recurrent Unit)

GRU is a **simpler version** of LSTM with only **two gates**:

1. **Update Gate:** Combines the functionality of LSTM's forget and input gates
2. **Reset Gate:** Determines how much past information to forget

Aspect	LSTM	GRU
Number of gates	3 (forget, input, output)	2 (update, reset)
Complexity	More complex	Simpler
Speed	Slower	Faster
Performance	Better on complex data	Often similar to LSTM
When to use	Complex sequences	Simpler data, when speed matters

LSTM is an upgrade of RNN, just as CNN is an upgrade of ANN. GRU is a simplified version of LSTM.

7.5 Smart Memory Demonstration

With gated memory (LSTM/GRU), the memory update formula preserves information across steps. Unlike basic RNN where tanh causes exponential decay, the gated mechanism maintains stable memory values:

- No vanishing gradient observed in the memory cell
- No exploding gradient observed
- Memory values remain stable across multiple time steps

This is because the gates learn to selectively pass or block information, rather than applying a uniform

transformation at every step.

8 Summary

Concept	Basic RNN	LSTM/GRU
Memory type	Simple memory	Smart (gated) memory
Long-term dependency	Fails	Captured effectively
Memory control	No control	Selective remember/forget
Gradient behavior	Vanishes or explodes	Stabilized via gates
Update mechanism	Uniform tanh	Controlled gate-based

- Basic RNN implements a simple hidden state update: $h_t = \tanh(W_{hx}x_t + W_{hh}h_{t-1} + b)$
- This leads to three core problems: **memory compression**, **memory fading**, and **gradient problems** (vanishing and exploding)
- The vanishing gradient problem occurs because multiplying values between -1 and 1 repeatedly drives the gradient toward zero
- The exploding gradient problem occurs when gradient values greater than 1 multiply and grow unboundedly
- LSTM introduces a memory cell with three gates (forget, input, output) to control memory flow
- GRU is a simpler alternative with two gates (update, reset), often performing comparably to LSTM
- The gated memory mechanism behaves like a skip connection, preventing memory from fading
- Basic RNN is not sufficient because long-term dependencies are not captured